SPLITSTACK

SplitStack Refurbishment - Prompt Log

All prompts used to analyse, plan and rebuild SplitStack, plus the planned prompts that complete the project.

Student: Mahaprabhu K
Assistant: Claude Code (Claude Opus 5.5)
Repository: https://github.com/MahaprabhuK/SplitStack
Executed prompts: 15
Planned prompts: 15
Last updated: 2026-10-15

How to read this log

Part A lists every step already carried out, in order. "User prompt" entries are the student's own messages, quoted verbatim; "Working prompt" entries are the instructions executed for that step, written so they can be reused. Part B lists the planned prompts that complete Review 2, Review 3 and the wrap-up; they have not been run yet and are marked "Planned prompt".

Stages

Stage	Scope	Prompts
Planning	Analysis of the existing project, audit report review, gap analysis and refurbishment plan	4
Phase 0	Repository setup, hygiene and documentation tooling	2
Review 1	HTML + CSS - static, semantic pages and a token-based design system	6
Review 2	JavaScript - vanilla ES modules, interactivity, client-side algorithms	8
Review 3	Node.js + MongoDB - Express REST API, Mongoose models, authentication, OCR	6
Wrap-up	Documentation, cleanup and deployment	4

Summary

#	Stage	Title	Type	Status
1	Planning	Initial request from the student	User prompt	Done
2	Planning	Analyse the repository	Working prompt	Done
3	Planning	Analyse the audit report	Working prompt	Done
4	Planning	Gap analysis and refurbishment plan	Working prompt	Done
5	Phase 0	Branching and repository hygiene	Working prompt	Done
6	Phase 0	Prompt-log tooling (this document)	Working prompt	Done
7	Review 1	CSS design system with cascade layers	Working prompt	Done
8	Review 1	Static semantic HTML pages (25 pages)	Working prompt	Done
9	Review 1	Local static server for previewing the client	Working prompt	Done
10	Review 1	Validate HTML and fix issues	Working prompt	Done
11	Review 1	Visual QA and layout bug fixes	Working prompt	Done
12	Review 1	Client README and Review 1 checkpoint commit	Working prompt	Done
13	Wrap-up	Request: GitHub repository, website link and PDF of prompts	User prompt	Done
14	Wrap-up	PDF export of the prompt log, including planned prompts	Working prompt	Done
15	Wrap-up	Create the GitHub repository and publish the site	Working prompt	Done
P1	Review 2	JavaScript architecture and app shell	Planned prompt	Planned
P2	Review 2	Client data layer backed by localStorage	Planned prompt	Planned
P3	Review 2	Render pages from data	Planned prompt	Planned
P4	Review 2	Forms, validation and CRUD flows	Planned prompt	Planned
P5	Review 2	Group splitting and the debt-simplification algorithm	Planned prompt	Planned

#	Stage	Title	Type	Status
P6	Review 2	Charts, budgets, forecast, insights, search and notifications	Planned prompt	Planned
P7	Review 2	Shopping checklist and receipt scan preview	Planned prompt	Planned
P8	Review 2	Review 2 QA, log and commit	Planned prompt	Planned
P9	Review 3	Express server and MongoDB connection	Planned prompt	Planned
P10	Review 3	Mongoose data models	Planned prompt	Planned
P11	Review 3	Authentication and REST API	Planned prompt	Planned
P12	Review 3	Receipt upload and server-side OCR	Planned prompt	Planned
P13	Review 3	Connect the client to the API	Planned prompt	Planned
P14	Review 3	Seed data and automated tests	Planned prompt	Planned
P15	Wrap-up	Remove legacy code, document and deploy	Planned prompt	Planned

Part A - Executed prompts

Planning - Analysis of the existing project, audit report review, gap analysis and refurbishment plan

#1 Initial request from the student

User prompt | 2026-10-14

PROMPT

```
i have a project in my web technology course and we have made a website. i want to check everything and refurbish if needed. the project shd have these things. html + css, javascript, node js , mangodbs. for review 1-html+css, for review 2 javascript. for review 3 the rest two. i have attached a report of the existing project. i want you to analyse the repositry and then the audit report. and give a good plan for the refurshing if needed. during refurbishing i want you to document the prompts you'll use to build the project save it as a log document(word)
```

PURPOSE

Defines the goal: verify the project meets the course stack (HTML+CSS, JavaScript, Node.js, MongoDB) across three reviews, refurbish it, and log all prompts in a Word document.

FILES

SplitStack_Comprehensive_Technical_Audit.pdf (input)

RESULT

Task accepted; analysis started.

VERIFICATION

-

#2 Analyse the repository

Working prompt | 2026-10-14

PROMPT

```
Inventory the SplitStack repository: list every tracked file, read the git history, the dependency manifests (requirements.txt / package.json), the base HTML layout, the CSS design tokens and all JavaScript (main.js plus inline <script> blocks in templates). Identify the actual technology stack and compare it with the course requirement of HTML+CSS, JavaScript, Node.js and MongoDB.
```

PURPOSE

Establish ground truth about what the code really is, independent of the audit report.

FILES

app.py, requirements.txt, templates/**, static/css/style.css, static/js/main.js, .gitignore

RESULT

Stack is Python 3 + Flask 3.1 + SQLite with Jinja2 templates; no Node.js, no MongoDB, no package.json. Frontend JS is ~140 lines of jQuery. 25 Jinja templates cannot render without Flask. 4 user-uploaded receipt images are committed to git.

VERIFICATION

File listing, git log, grep for <script> tags, manual reading of base.html and main.js.

#3 Analyse the audit report

Working prompt | 2026-10-14

PROMPT

```
Extract the text of SplitStack_Comprehensive_Technical_Audit.pdf (61 pages) and summarise its findings: technology stack, architecture, route inventory, database schema, security findings (SEC-01..SEC-07), test results, technical debt and the five-phase refurbishment roadmap (Phases A-E). Note anything the audit does not cover relative to the course requirements.
```

PURPOSE

Reuse the audit's verified findings and identify its blind spots.

FILES

SplitStack_Comprehensive_Technical_Audit.pdf

RESULT

Audit confirms 48 routes, 7 blueprints, 15 SQLite tables, 47/47 tests passing. Key findings kept: incomplete light theme, --text-dim contrast 3.8:1 (fails WCAG AA), tables scroll on mobile, search/notification bell not wired, no rate limiting, no security headers, ephemeral upload storage. Blind spot: the audit never mentions that Node.js and MongoDB are required, so its Flask-specific roadmap (SQLite WAL, Flask-Limiter, PostgreSQL) does not satisfy Review 3.

VERIFICATION

Text extracted with pypdf; sections 2, 12-17 and Volume VI read.

#4 Gap analysis and refurbishment plan

Working prompt | 2026-10-14

PROMPT

```
Produce a refurbishment plan that maps the work onto the three course reviews: Review 1 must be demonstrable with only HTML+CSS, Review 2 must add substantial vanilla JavaScript on top of the same pages, and Review 3 must replace the backend with Node.js (Express) and MongoDB while keeping every existing feature. Fold in the still-relevant audit findings. Ask the student only about decisions that are genuinely theirs: backend strategy, where MongoDB runs, feature scope, and whether to keep jQuery.
```

PURPOSE

Turn the analysis into an ordered plan with clear review checkpoints.

FILES

docs/REFURBISH_PLAN.md

RESULT

Plan: Phase 0 setup -> Review 1 static pages + design system -> Review 2 vanilla JS with a localStorage data layer -> Review 3 Express + Mongoose REST API, with the frontend data layer switched to fetch(). Student's answers: 'do the most optimal method' for the backend (=> replace Flask with Node+MongoDB, keep Flask on a legacy branch), MongoDB Atlas free tier, keep every feature including OCR (via tesseract.js), vanilla JS modules instead of jQuery. Student also asked: 'just make sure you log everything'.

VERIFICATION

Decisions confirmed through a multiple-choice question in chat.

Phase 0 - Repository setup, hygiene and documentation tooling

#5 Branching and repository hygiene

Working prompt | 2026-10-14

PROMPT

```
Preserve the current Flask application on a local 'legacy' branch and create a 'refurbish'
working branch. Stop tracking the four user-uploaded receipt images (hash-named files in
uploads/receipts/) while keeping the test_invoice_* samples, deduplicate .gitignore, and add
Node-specific ignores (node_modules, coverage, .env). Create the client/ folder structure
(pages, css, js, assets).
```

PURPOSE

Protect the original work, remove private data from version control, and prepare the new layout.

FILES

.gitignore, uploads/receipts/* (untracked), client/

RESULT

Branches legacy and refurbish created; 4 receipts untracked (still on disk); .gitignore rewritten.

VERIFICATION

git status shows the 4 receipts as deleted from the index and no longer as untracked files.

#6 Prompt-log tooling (this document)

Working prompt | 2026-10-14

PROMPT

```
Create a Word prompt log that is easy to keep up to date: store every prompt as an entry in
docs/prompt_log.json (id, date, stage, title, type, prompt, purpose, files, result,
verification) and write a Node script docs/tools/build-prompt-log.js using the 'docx' npm
package that renders a title page, a stage overview, a summary table of all entries, and one
detailed section per entry with the prompt shown verbatim. Add an 'npm run log' script.
```

PURPOSE

Satisfy the requirement to document every prompt as a Word document, regenerated after each step.

FILES

package.json, docs/prompt_log.json, docs/tools/build-prompt-log.js, docs/SplitStack_Prompt_Log.docx

RESULT

Log generator working; document regenerated after every subsequent step.

VERIFICATION

npm run log produces the .docx; contents read back with python-docx.

Review 1 - HTML + CSS - static, semantic pages and a token-based design system

#7 CSS design system with cascade layers

Working prompt | 2026-10-14

PROMPT

```
Rebuild SplitStack's stylesheet as a token-based design system split into files and ordered with CSS cascade layers: tokens.css (colours, type scale, spacing, radii, shadows; dark default plus a light theme via prefers-color-scheme and a [data-theme] attribute for later JavaScript), base.css (reset, typography, focus ring, skip link, visually-hidden), layout.css (app shell with sticky sidebar and topbar, grids, auth and landing layouts, responsive breakpoints at 1100/900/640px, CSS-only mobile drawer using a hidden checkbox and :has()), components.css (cards, buttons, badges, category tags via a --cat custom property, stat cards, budget meters, forms with :user-invalid feedback, segmented radios, switches, tables that become stacked cards under 640px using data-label, alerts, empty states, lists, CSS-only bar and conic-gradient doughnut charts, transfer cards, checklist, file drop zone), pages.css and utilities.css. main.css imports them with @layer. Fix the audit findings: muted text must reach at least 4.5:1 contrast, the light theme must be complete, and tables must not scroll sideways on mobile.
```

PURPOSE

Review 1 requires a clean, well-structured CSS codebase; this also fixes audit items SEC-06 (contrast), B.1 (light theme) and B.2 (responsive tables).

FILES

client/css/main.css, client/css/tokens.css, client/css/base.css, client/css/layout.css, client/css/components.css, client/css/pages.css, client/css/utilities.css

RESULT

Six layered stylesheets. --text-dim raised from #64748b (3.8:1) to #8794a9 (about 5.6:1) on dark and #5d6879 on light surfaces.

VERIFICATION

Rendered in the browser at 1366px and 375px; computed styles inspected.

#8 Static semantic HTML pages (25 pages)

Working prompt | 2026-10-14

PROMPT

```
Convert every Jinja template of the Flask app into a standalone static HTML5 page under client/ that opens without a server-side framework, using realistic sample data in Indian rupees for one demo user (Priya Raman) as of 28 Oct 2026. Pages: landing (index.html), login, register, 404, dashboard, analytics, expenses, add expense, edit expense, income, log income, budgets, recurring payments, categories, groups, create group, group detail, add shared expense, settle up, shopping lists, new list, list detail, receipt scan, receipt confirm, profile. Every app page shares the same sidebar, topbar (search, notifications, avatar) and footer. Use semantic landmarks (header, nav, main, aside, section, footer), a skip link, breadcrumbs, labelled forms with HTML5 validation (required, min, max, step, pattern, type=email/date/number), tables with captions and scope, time elements, aria-current for the active link, role=progressbar with values for budget meters, and a text alternative for every chart. Keep the sample numbers internally consistent (category totals, group balances, the debt-simplification plan). Avoid inline styles except CSS custom properties such as --value or --cat. Generate the repeated layout with a throwaway script so that the committed output is plain HTML.
```

PURPOSE

Deliver the Review 1 HTML+CSS milestone: every screen of SplitStack, demonstrable with only HTML and CSS.

FILES

client/index.html, client/pages/*.html (24 pages), client/assets/favicon.svg, client/assets/sample-receipt.webp

RESULT

25 static pages. CSS-only interactions: mobile drawer (checkbox + :has), equal/custom split toggle (:has(#split-custom:checked)), purchased items struck through (:has(input:checked)), click-or-drop file input.

VERIFICATION

See the validation entry.

#9 Local static server for previewing the client

Working prompt | 2026-10-14

PROMPT

```
Add a dependency-free Node static file server (scripts/serve-client.js) that serves client/ on http://localhost:5500, refuses paths outside client/, falls back to pages/404.html, and is started with "npm run client". Opening pages straight from disk also works, but Review 2 ES modules will need an http:// origin.
```

PURPOSE

Preview the site reliably now and prepare for JavaScript modules in Review 2.

FILES

scripts/serve-client.js, package.json, .claude/launch.json

RESULT

npm run client serves the site.

VERIFICATION

Pages loaded in the in-app browser at localhost:5500.

#10 Validate HTML and fix issues

Working prompt | 2026-10-14

PROMPT

```
Install html-validate as a dev dependency with a project config (.htmlvalidate.json) based on html-validate:recommended, run it on client/**/*.html and fix every error: unescaped ampersands, a search form without a submit button, unnamed duplicate landmarks, aria-label on a plain div, a form without a submit button, redundant ARIA roles. Allow shared names for checkbox groups and the custom progressbar (needed for over-100% budget states). Then check that every relative href/src resolves to a real file.
```

PURPOSE

Show that the Review 1 markup is standards-compliant and accessible.

FILES

.htmlvalidate.json, package.json, client/**/*.html

RESULT

First run: 100 errors. After fixes: 0 errors across 25 pages; 0 broken internal links.

VERIFICATION

npx html-validate "client/**/*.html" exits 0.

#11 Visual QA and layout bug fixes

Working prompt | 2026-10-14

PROMPT

```
Open the pages in the browser at desktop (1366px) and mobile (375px) widths, look for layout bugs and fix them in the CSS. Then load every page in a 375px and a 1280px iframe and report any element that overflows the viewport horizontally.
```

PURPOSE

Catch rendering problems that a validator cannot see.

FILES

client/css/layout.css, client/css/components.css

RESULT

Fixed: (1) the scrim <label> occupied a grid cell on desktop and pushed the main column below the sidebar; (2) the mobile menu button showed on desktop and the search icon overlapped the placeholder, because the components layer overrides the layout layer; those elements now have layout-only classes; (3) table dates wrapped onto two lines.

VERIFICATION

Screenshots at both widths; scripted overflow check across all pages.

#12 Client README and Review 1 checkpoint commit

Working prompt | 2026-10-14

PROMPT

```
Write client/README.md explaining how to view the site (npm run client), the folder structure, and a table of the CSS techniques used, with the file each one is in, so the Review 1 demo can point to them. Add an "npm run lint:html" script. Re-run validation, regenerate the prompt log, and commit Review 1 on the refurbish branch.
```

PURPOSE

Make Review 1 easy to present and record a clean checkpoint.

FILES

client/README.md, package.json, docs/SplitStack_Prompt_Log.docx

RESULT

Review 1 committed. Next: Review 2 (vanilla JavaScript modules).

VERIFICATION

npm run lint:html passes; git log shows the Review 1 commit.

Wrap-up - Documentation, cleanup and deployment

#13 Request: GitHub repository, website link and PDF of prompts

User prompt | 2026-10-15

PROMPT

```
create a new repositry for this project in my gihub , and after than give me the github link and then the website link and also a pdf containing the prompts to complete project.
```

PURPOSE

Publish the project under the student's own GitHub account, host the site, and export the prompt log (including the remaining planned prompts) as a PDF.

RESULT

Done: see the following entries.

VERIFICATION

-

#14 PDF export of the prompt log, including planned prompts

Working prompt | 2026-10-15

PROMPT

```
Export the prompt log to PDF with a Python reportlab script (docs/tools/build_prompt_pdf.py)
that reads docs/prompt_log.json: a title page, a stage overview table, a summary of every
entry, Part A with every executed prompt verbatim in a shaded box with purpose, files, result
and verification, and Part B with the planned prompts that complete Review 2, Review 3 and the
wrap-up, clearly marked as not yet run. Add the planned prompts to prompt_log.json and to the
Word log as well, and make "npm run log" rebuild both documents.
```

PURPOSE

Give the student a PDF containing the prompts needed to complete the whole project.

FILES

docs/tools/build_prompt_pdf.py, docs/tools/build-prompt-log.js, docs/prompt_log.json, docs/SplitStack_Prompt_Log.pdf, docs/SplitStack_Prompt_Log.docx, package.json

RESULT

PDF with the executed prompts and 15 planned prompts; the Word log has the same content.

VERIFICATION

Pages rendered to images and inspected; text extracted with pypdf.

#15 Create the GitHub repository and publish the site

Working prompt | 2026-10-15

PROMPT

```
Install the official GitHub CLI with winget and sign in through the device-code flow (the
student approves it in the browser). Before publishing, check that no secrets or databases are
tracked, review the receipt images that are still in the commit history (they are generic
sample invoices), and confirm with the student that a public repository is acceptable because
the team's original repository is private. Then create the public repository
MahaprabhuK/SplitStack, push the refurbish branch as main together with the legacy branch, add
a GitHub Actions workflow (.github/workflows/pages.yml) that deploys the client/ folder to
GitHub Pages, enable Pages with the "GitHub Actions" source, and confirm that the site
responds.
```

PURPOSE

Give the student a repository link and a live website link.

FILES

.github/workflows/pages.yml, README.md

RESULT

Repository: https://github.com/MahaprabhuK/SplitStack - Website: https://mahaprabhuk.github.io/SplitStack/

VERIFICATION

gh repo view; Pages workflow run; HTTP check of the live URL.

Part B - Planned prompts to complete the project

Run these in order. Each builds on the previous one; after each, add a log entry and rebuild this document.

Review 2 - JavaScript - vanilla ES modules, interactivity, client-side algorithms

P1 JavaScript architecture and app shell

Planned prompt

PROMPT

```
Add vanilla JavaScript to the Review 1 pages as native ES modules loaded with <script
type="module" src="../js/app.js">. Create client/js/ with: app.js (entry; detects the page
from <body data-page> and calls that page's init), core/dom.js (qs, qsa, on, delegate,
createEl helpers), core/format.js (formatINR with Intl.NumberFormat("en-IN", {style:
"currency", currency: "INR"}), date formatting), ui/theme.js (dark/light/system toggle stored
in localStorage and applied to <html data-theme> by a tiny inline script in <head> so there is
no flash), ui/nav.js (replace the CSS checkbox drawer with a real <button aria-expanded
aria-controls>, close on Escape, scrim click and link click, keep focus inside while open),
ui/toast.js (accessible role=status toasts) and ui/confirm.js (a <dialog> element replacing
window.confirm for every delete form). No jQuery and no frameworks.
```

PURPOSE

Lay the foundation for Review 2 with a clean module structure.

FILES

client/js/app.js, client/js/core/*.js, client/js/ui/*.js, client/pages/*.html

P2 Client data layer backed by localStorage

Planned prompt

PROMPT

```
Create client/js/data/api.js exposing async functions with the same shape the future REST API
will have: auth (register, login, logout, currentUser), expenses and income CRUD with
filtering, sorting and pagination, categories, budgets, recurring payments, groups (members,
shared expenses, balances, settlements), shopping lists and items, and analytics summaries.
Back it with client/js/data/store.js, which keeps collections in localStorage under a
versioned key, generates ids, and seeds the same sample data used in Review 1 (Priya Raman,
October 2026) on first run. Every function returns a Promise and validates its input, throwing
errors with a field and a message, so Review 3 can swap the implementation to fetch() without
touching page code.
```

PURPOSE

Make the whole app genuinely interactive before a backend exists.

FILES

client/js/data/api.js, client/js/data/store.js, client/js/data/seed.js

P3 Render pages from data

Planned prompt

PROMPT

```
Replace the hard-coded sample rows in the dashboard, expenses, income, budgets, recurring, categories, groups, group detail, settlements, shopping and profile pages with markup rendered by JavaScript from api.js, using <template> elements and textContent (never innerHTML with user data). Keep the static markup as the no-JavaScript fallback. Implement the expenses filter bar (search, category, method, date range, sort) with debounced live filtering and query-string sync, pagination, and empty states.
```

PURPOSE

Turn the static pages into working screens.

FILES

client/js/pages/*.js, client/pages/*.html

P4 Forms, validation and CRUD flows

Planned prompt

PROMPT

```
Wire every form (register, login, expense add/edit, income, budget, recurring, category, group create, add member, shared expense, shopping list and item, profile, change password) to api.js. Use the Constraint Validation API plus custom rules (passwords match, date not in the future, budget greater than zero) with inline error messages linked by aria-describedby, and move focus to the first invalid field. Show a toast after each save and redirect afterwards (Post/Redirect/Get). Add password show/hide toggles with aria-pressed. Protect app pages: redirect to login.html when nobody is signed in.
```

PURPOSE

Cover the core JavaScript skills examiners look for: events, DOM manipulation, validation.

FILES

client/js/pages/*.js, client/js/ui/forms.js

P5 Group splitting and the debt-simplification algorithm

Planned prompt

PROMPT

```
In client/js/domain/splits.js implement equal splits in integer paise with the leftover paise assigned to the payer, and custom splits that must sum exactly to the total. On the shared-expense form show a live "remaining to assign" counter and disable submit until it reaches zero. In client/js/domain/settle.js implement net balances (paid minus share) and the greedy minimum-cash-flow algorithm: repeatedly match the largest debtor with the largest creditor and transfer min(debt, credit) until every balance is within one paisa; return the transfer list and the number of original pairwise debts. Render the settle-up page from it, and make "Mark paid" record a settlement and recompute balances. Add unit tests runnable with node --test using the Goa Trip example (balances +1700, -2500, +5700, -4900 give 3 transfers).
```

PURPOSE

Show algorithmic JavaScript, the core idea of SplitStack.

FILES

client/js/domain/splits.js, client/js/domain/settle.js, tests/client/*.test.js

P6 Charts, budgets, forecast, insights, search and notifications

Planned prompt

PROMPT

```
Render the dashboard and analytics charts with Chart.js 4 loaded from a pinned CDN URL, reading colours from the CSS custom properties so both themes work, and keep the CSS charts as a fallback. Compute budget status (ok under 75%, caution 75-90%, warning 90-100%, danger over 100%) and update the meters. Port the spending forecast (average daily spend multiplied by the days in the month) to client/js/domain/forecast.js. Generate smart insights from the data (over-budget categories, month-over-month change, upcoming recurring bills). Make the topbar search show live results across expenses, groups and lists in a listbox with keyboard navigation, and make the bell open a notifications panel built from the insights with an unread count.
```

PURPOSE

Finish the interactive features that were disconnected in the original app.

FILES

client/js/pages/dashboard.js, client/js/pages/analytics.js, client/js/domain/forecast.js, client/js/ui/search.js, client/js/ui/notifications.js

P7 Shopping checklist and receipt scan preview

Planned prompt

PROMPT

```
Make the shopping checklist toggle items with optimistic UI, edit actual prices inline, recompute the estimated, actual and variance totals live, and convert the list to an expense only once (prevent duplicates). On the receipt scan page add drag-and-drop with dragenter/dragleave styling, client-side checks (PNG, JPEG or WEBP, up to 5 MB) and an image preview via URL.createObjectURL. Until the backend exists, run OCR in the browser with tesseract.js from a pinned CDN, parse the merchant, date and total with regular expressions (prefer the line containing TOTAL), show progress, and pre-fill the confirm form.
```

PURPOSE

Complete the remaining interactive flows.

FILES

client/js/pages/shopping-detail.js, client/js/pages/receipt-scan.js, client/js/domain/receipt-parser.js

P8 Review 2 QA, log and commit

Planned prompt

PROMPT

```
Run ESLint (recommended rules, browser globals, ES2023 modules) over client/js and fix every finding; run the domain unit tests; test every page manually at 1366px and 375px in dark and light themes, using only the keyboard, and with JavaScript disabled (pages must still show their static content). Update client/README.md with the JavaScript architecture and a table of the JS concepts used and where. Add each step to docs/prompt_log.json, rebuild the Word and PDF logs, and commit "Review 2".
```

PURPOSE

Close Review 2 with evidence.

FILES

eslint.config.js, client/README.md, docs/*

Review 3 - Node.js + MongoDB - Express REST API, Mongoose models, authentication, OCR

P9 Express server and MongoDB connection

Planned prompt

PROMPT

```
Create server/ as a Node.js 20+ project (ES modules) with Express: server/src/app.js (helmet with a Content Security Policy that allows only the CDNs in use, compression, express.json, express-session stored with connect-mongo, express-rate-limit on auth and upload routes, a CSRF token check on state-changing requests, and a central error handler returning JSON {error, fields}), server/src/index.js (connect to MongoDB Atlas with Mongoose using MONGODB_URI from server/.env, then listen), and static serving of client/. Add server/.env.example (MONGODB_URI, SESSION_SECRET of at least 32 characters, PORT) and npm scripts dev (node --watch) and start. Never commit .env.
```

PURPOSE

Start Review 3 with a secure, conventional Express setup.

FILES

server/package.json, server/src/app.js, server/src/index.js, server/.env.example

P10 Mongoose data models

Planned prompt

PROMPT

```
Design Mongoose schemas with validation and indexes to replace the 15 SQLite tables: User (name, email unique and lowercase, passwordHash, preferences), Category (user, or null for defaults; name; type expense or income; icon; color), Expense and Income (user, category, amount stored as integer paise, date, description or source, paymentMethod), Budget (user, category or null, month, year, limit; unique on user+category+month+year), Recurring (name, amount, frequency, startDate, nextDueDate, active), Group (name, description, createdBy, members [{user, role admin|member, joinedAt}]), SharedExpense (group, title, amount, paidBy, category, date, splitType, shares [{user, amount}] with a validator that the shares sum to the amount), Settlement (group, payer, payee, amount, status, completedAt), ShoppingList (owner, optional group, name, budget, items [{name, quantity, estimatedPrice, actualPrice, purchased, assignedTo}], convertedExpense), Receipt (user, file reference, merchant, date, amount, rawText, confidence) and Insight (user, kind, message, read). Explain in comments which data is embedded and which is referenced, and why.
```

PURPOSE

Show MongoDB data modelling, the core of Review 3.

FILES

server/src/models/*.js

P11 Authentication and REST API

Planned prompt

PROMPT

```
Implement the REST API under /api with one express.Router per module, mirroring the 48 legacy Flask routes: /api/auth (register with bcrypt cost 12, login, logout, me), /api/expenses, /api/income, /api/categories, /api/budgets, /api/recurring (including toggle), /api/groups (members, shared expenses, balances, settlements, leave), /api/shopping (lists, items, toggle, convert to expense), /api/analytics (monthly, categories, forecast, insights) and /api/search. Validate every input with a schema (zod or express-validator), check ownership or group membership on every request, use MongoDB transactions for multi-document writes (recording a settlement, converting a list), and reuse the Review 2 settle.js and splits.js modules on the server so the algorithm has a single implementation.
```

PURPOSE

Deliver the Node.js back end with a complete, secure API.

FILES

server/src/routes/*.js, server/src/services/*.js, server/src/middleware/*.js

P12 Receipt upload and server-side OCR

Planned prompt

PROMPT

```
Add POST /api/receipts using multer with memory storage, a 5 MB limit and magic-byte checks for PNG, JPEG and WEBP; preprocess with sharp (rotate from EXIF, greyscale, upscale small images, normalise); run tesseract.js with one shared worker; parse the merchant, date and total with the Review 2 parser; store the image in MongoDB GridFS (so it survives redeploys on free hosting) and the parsed fields in Receipt. GET /api/receipts/:id/image streams the image only to its owner. Confirming creates the expense; nothing becomes an expense without confirmation.
```

PURPOSE

Port the OCR feature to Node with persistent storage.

FILES

server/src/routes/receipts.js, server/src/services/ocr.js

P13 Connect the client to the API

Planned prompt

PROMPT

```
Re-implement client/js/data/api.js with fetch() against /api, sending credentials and the CSRF token header, mapping HTTP errors to the same {field, message} errors the pages already handle, and showing a toast when offline. Remove the localStorage store except for the theme preference. Because the page code depends only on api.js, no page module should need changes; confirm this by running every flow end to end against a local server connected to Atlas.
```

PURPOSE

Finish the full-stack integration.

FILES

client/js/data/api.js

P14 Seed data and automated tests

Planned prompt

PROMPT

```
Add server/scripts/seed.js that creates the demo user, categories, six months of expenses and income, budgets, recurring payments, the three groups with shared expenses, and shopping lists. It must be idempotent and must refuse to run against a database whose name does not end in -dev or -test unless --force is passed. Write Jest + Supertest tests using mongodb-memory-server for auth, ownership checks, CSRF rejection, rate limiting, split validation, the settlement algorithm, budget status, convert-to-expense duplicate prevention and receipt upload validation, covering what the 47 legacy Python tests checked. Add an npm test script and a GitHub Actions workflow that runs lint and tests on every push.
```

PURPOSE

Prove the back end works and keep it working.

FILES

server/scripts/seed.js, server/tests/*.test.js, .github/workflows/ci.yml

Wrap-up - Documentation, cleanup and deployment

P15 Remove legacy code, document and deploy

Planned prompt

PROMPT

```
Delete the Flask app (app.py, config.py, database/, modules/, templates/, static/, ml/, tests/*.py, requirements.txt, Procfile) from main, since it is kept on the legacy branch. Rewrite README.md: features, the stack per review (HTML+CSS, JavaScript, Node.js+MongoDB), setup steps, environment variables, a data-model diagram, the API table and screenshots. Deploy the full app to Render as a Node web service (build: npm ci; start: npm start; environment: MONGODB_URI, SESSION_SECRET, NODE_ENV=production) with MongoDB Atlas network access configured, and keep the static site on GitHub Pages. Finish the prompt log, export it to Word and PDF, and tag the release v3.0.0.
```

PURPOSE

Deliver a clean, documented, deployed project.

FILES

README.md, render.yaml, docs/*